



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura:

Estructura de Datos y Algoritmos II

Proyecto final:

Software para la visualización y estudio de algoritmos y estructuras de datos
-Backend-

Documentación Técnica del Módulo API

Análisis, Diseño e Implementación de Endpoints REST en el
Backend

Carpeta documentada:

app/api/

Equipo de Desarrollo de Backend

18 de mayo de 2026

1. Módulo API

La carpeta `api` contiene los endpoints encargados de comunicar el front-end con la lógica interna del backend. Su función principal es recibir datos enviados desde la interfaz, llamar a los algoritmos correspondientes y devolver respuestas en formato JSON.

Este módulo no implementa directamente los algoritmos principales, sino que actúa como una capa intermedia entre el usuario y la carpeta `algoritmos`. Por ello, su documentación se enfoca en describir qué hace cada archivo, qué rutas expone y cómo pueden ser utilizadas por el front-end.

1.1. Estructura General

La carpeta `api` contiene los siguientes archivos principales:

```
api/  
  files.py  
  graphs.py  
  searching.py  
  sorting.py
```

El archivo `__init__.py` no se considera en esta documentación debido a que se encuentra vacío.

1.2. Archivo `files.py`

El archivo `files.py` contiene endpoints relacionados con la carga y consulta de archivos dentro del backend.

Endpoint	Método	Descripción
<code>/create</code>	POST	Recibe un archivo enviado desde el front-end y lo guarda en la carpeta de almacenamiento del backend.
<code>/info</code>	GET	Recibe el nombre de un archivo y devuelve información como nombre, tamaño y fecha de creación.

Cuadro 1: Endpoints del archivo `files.py`

Desde el front-end, este archivo puede usarse para implementar una vista de carga de archivos. El usuario selecciona un archivo, lo sube al sistema y posteriormente puede consultar sus datos principales.

Una interfaz visual podría incluir:

- Botón para seleccionar archivo.
- Botón para subir archivo.
- Mensaje de confirmación.
- Tabla o tarjeta con información del archivo.

1.3. Archivo `graphs.py`

El archivo `graphs.py` contiene endpoints para representar grafos y ejecutar recorridos BFS y DFS.

Endpoint	Método	Descripción
<code>/represent</code>	POST	Recibe nodos y aristas, y devuelve la matriz de adyacencia y la lista de adyacencia del grafo.
<code>/bfs</code>	POST	Ejecuta búsqueda en anchura sobre el grafo recibido y devuelve el orden de recorrido.
<code>/dfs</code>	POST	Ejecuta búsqueda en profundidad sobre el grafo recibido y devuelve el orden de recorrido.

Cuadro 2: Endpoints del archivo `graphs.py`

El modelo de entrada usado por estos endpoints recibe:

- **edges**: lista de aristas.
- **n**: número total de nodos.
- **inicio**: nodo inicial para el recorrido.

En el front-end, esta sección puede representarse mediante una vista donde el usuario ingrese nodos y aristas. Después, puede seleccionar si desea ver la representación del grafo, ejecutar BFS o ejecutar DFS.

Una interfaz visual podría incluir:

- Campo para número de nodos.
- Campo para aristas.
- Selector de operación.
- Campo para nodo inicial.

- Visualización del grafo.
- Lista del recorrido obtenido.

1.4. Archivo `searching.py`

El archivo `searching.py` expone endpoints para ejecutar algoritmos de búsqueda sobre arreglos numéricos.

Endpoint	Método	Descripción
<code>/linear</code>	POST	Ejecuta búsqueda lineal sobre un arreglo.
<code>/binary</code>	POST	Ejecuta búsqueda binaria sobre un arreglo ordenado.
<code>/hash</code>	POST	Ejecuta búsqueda mediante tabla hash.

Cuadro 3: Endpoints del archivo `searching.py`

El modelo de entrada recibe:

- **array**: arreglo de números enteros.
- **target**: valor que se desea buscar.

Estos endpoints devuelven información como si el valor fue encontrado, su índice, número de comparaciones o colisiones y complejidad aproximada.

En el front-end, esta sección puede funcionar como un comparador de búsquedas. El usuario ingresa un arreglo, selecciona el algoritmo y escribe el valor objetivo.

Una interfaz visual podría incluir:

- Campo para ingresar el arreglo.
- Campo para el valor objetivo.
- Selector de algoritmo.
- Resultado de búsqueda.
- Índice encontrado.
- Comparaciones, colisiones o complejidad.

1.5. Archivo `sorting.py`

El archivo `sorting.py` contiene endpoints para ejecutar algoritmos de ordenamiento y comparar su rendimiento.

Endpoint	Método	Descripción
<code>/counting-sort</code>	POST	Ordena un arreglo usando Counting Sort.
<code>/radix-sort</code>	POST	Ordena un arreglo usando Radix Sort.
<code>/merge-sort</code>	POST	Ordena un arreglo usando Merge Sort.
<code>/compare</code>	GET	Ejecuta los tres algoritmos con el mismo arreglo y devuelve una comparación de tiempo, comparaciones y complejidad.

Cuadro 4: Endpoints del archivo `sorting.py`

El modelo de entrada para los endpoints de ordenamiento recibe:

- **array**: arreglo de números enteros.

El endpoint `/compare` recibe el arreglo como cadena de texto separada por comas, por ejemplo:

5,3,8,1,9,2,7,4,6

En el front-end, esta parte puede utilizarse para construir un visualizador de ordenamientos. El usuario ingresa un arreglo, selecciona el algoritmo y observa el resultado. También puede usar la comparación para ver cuál algoritmo fue más rápido.

Una interfaz visual podría incluir:

- Campo para ingresar arreglo.
- Selector de algoritmo.
- Botón para ordenar.
- Visualización del arreglo original.
- Visualización del arreglo ordenado.
- Tabla comparativa de tiempo y complejidad.

2. Resumen General de Uso en Front-End

El módulo `api` permite que el front-end acceda a las funciones principales del backend mediante peticiones HTTP. Cada archivo representa una categoría funcional distinta.

Archivo	Función principal	Vista sugerida en front-end
<code>files.py</code>	Carga y consulta de archivos.	Vista para subir archivos y mostrar metadatos.
<code>graphs.py</code>	Representación y recorrido de grafos.	Vista para crear grafos y visualizar BFS/DFS.
<code>searching.py</code>	Ejecución de algoritmos de búsqueda.	Vista para buscar valores dentro de arreglos.
<code>sorting.py</code>	Ejecución y comparación de ordenamientos.	Vista para ordenar arreglos y comparar rendimiento.

Cuadro 5: Resumen visual del módulo API

El flujo general entre front-end y backend sería:

1. El usuario ingresa datos desde la interfaz.
2. El front-end envía una petición al endpoint correspondiente.
3. El endpoint recibe los datos y llama al algoritmo necesario.
4. El backend devuelve una respuesta JSON.
5. El front-end muestra el resultado de forma visual.

3. Conclusión

La carpeta `api` funciona como la capa de comunicación entre el front-end y los algoritmos del backend. Su propósito es exponer rutas claras para que la interfaz pueda ejecutar operaciones como subir archivos, representar grafos, realizar búsquedas y ordenar arreglos.

Esta documentación sintetizada permite que el equipo de front-end identifique rápidamente qué endpoint debe utilizar, qué datos debe enviar y cómo puede mostrar los resultados dentro de la interfaz del sistema.